

Projeto de Programação Orientada a Objetos

Objetivo:

Desenvolver um sistema acadêmico capaz de organizar cursos, disciplinas, turmas, alunos, professores, matrículas e notas, aplicando conceitos de POO.

Tecnologias

- Java
- Git/GitHub
- Draw.io
- Google Sites

Equipe Projeto Escola

Caio – Interfaces, Exceções e Validações

Gabriel da Cunha – Main, Menus, Google Sites e Documentação

Gabriel Ferreira – Curso, Disciplina e Conteúdo Educacional

Iago – Repositories, Services e Integração

Julio – Escola e Turma

Adrian – Pessoa, Aluno e Professor

Problema Identificado

- Grande volume de informações acadêmicas
- Dependência entre cursos, disciplinas, turmas e matrículas
- Possibilidade de inconsistências
- Dificuldade de consultas e acompanhamento

Necessidade:

- Centralizar informações
- Organizar relacionamentos
- Facilitar consultas
- Aplicar conceitos de POO

Curso



Disciplina



Turma

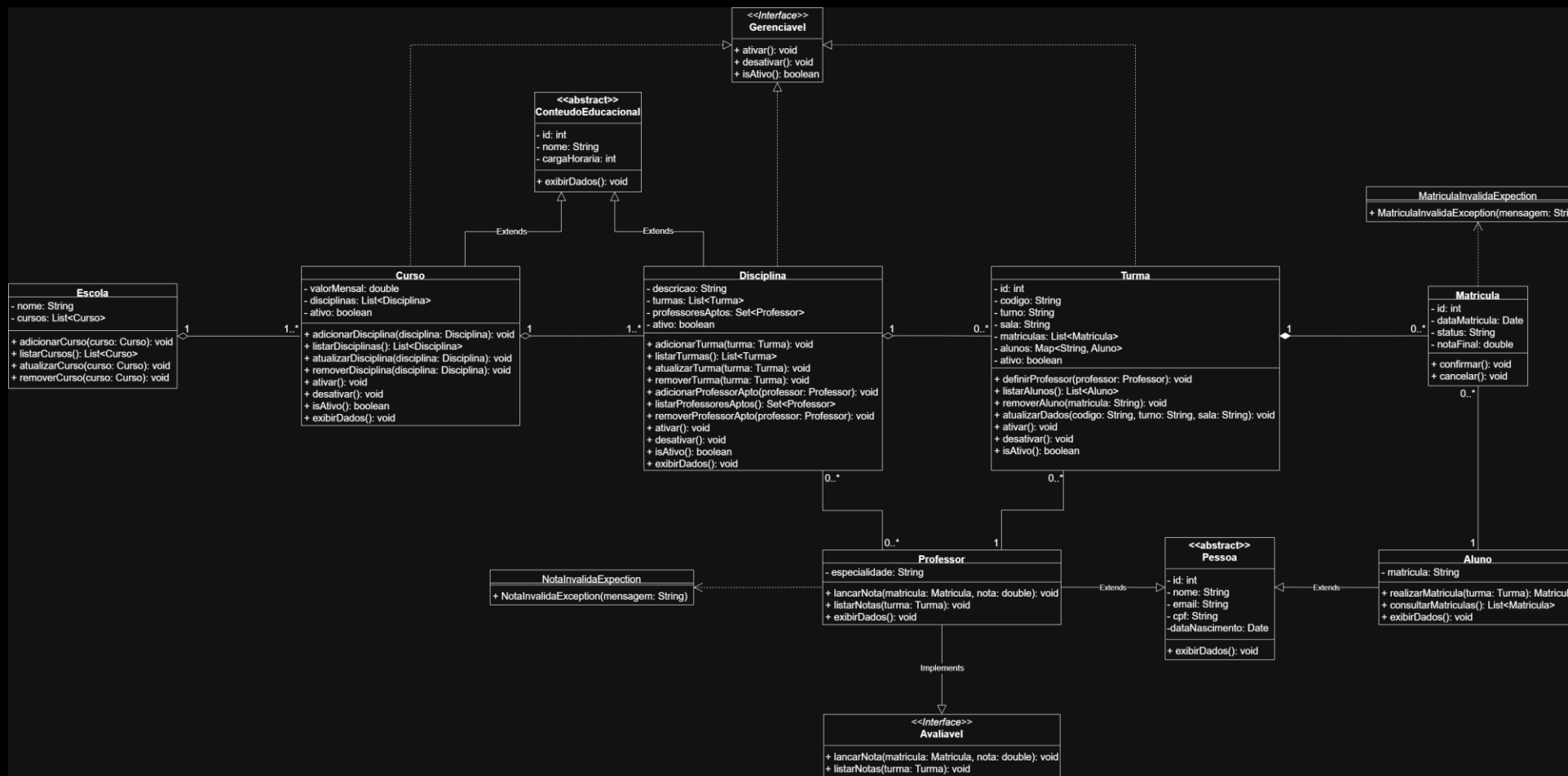


Matrícula



Aluno

Diagrama UML



Classes Abstratas

- Pessoa
- Conteúdo Educacional

Interfaces

- Gerenciavel
- Avaliavel

Entidades

- Escola
- Curso
- Disciplina
- Turma
- Professor
- Aluno
- Matricula

Exceções

- MatriculaInvalidaException
- NotaInvalidaException

Arquitetura do Sistema

Main
↓
Services
↓
Repositories
↓
Model

```
src/  
├ exception  
├ main  
├ model  
├ repository  
├ service  
└ util
```

Main → interação com usuário

Service → regras de negócio

Repository → armazenamento em memória

Model → entidades

Exception → tratamento de erros

Util → validações e auxiliares

Conceitos de Programação Orientada a Objetos

Herança



```
public class Aluno extends Pessoa {  
  
    private String matricula;  
  
    public Aluno(int id, String nome, String email, String cpf,  
                String dataNascimento, String matricula) {  
  
        super(id, nome, email, cpf, dataNascimento);  
        this.matricula = matricula;  
    }  
}
```

Interface



```
public class Professor extends Pessoa implements Avaliavel {  
  
    @Override  
    public void lancarNota(Matricula matricula, double nota) throws NotaInvalidaException { ...  
  
    @Override  
    public List<Matricula> listarNotas(Turma turma) { ...  
  
    @Override  
    public void exibirDados() { ...  
}  
}
```

Polimorfismo



```
private static void listarPessoasDoSistema() {  
    List<Pessoa> pessoas = new ArrayList<>();  
    pessoas.addAll(alunoService.listar());  
    pessoas.addAll(professorService.listar());  
  
    System.out.println(x: "\n--- Pessoas cadastradas no sistema ---");  
    for (Pessoa pessoa : pessoas) {  
        pessoa.exibirDados();  
    }  
}
```


Desafios Encontrados

- Modelagem UML
- Implementação da classe Matrícula
- Integração entre Main, Services e Repositories
- Ajustes após mudanças na modelagem
- Organização da arquitetura em camadas

Melhorias Futuras

- Persistência em banco de dados
- Interface gráfica
- Relatórios acadêmicos
- Controle de acesso

Obrigado!

Perguntas?

Professor Washington • Programação Orientada a
Objetos • Equipe Projeto Escola